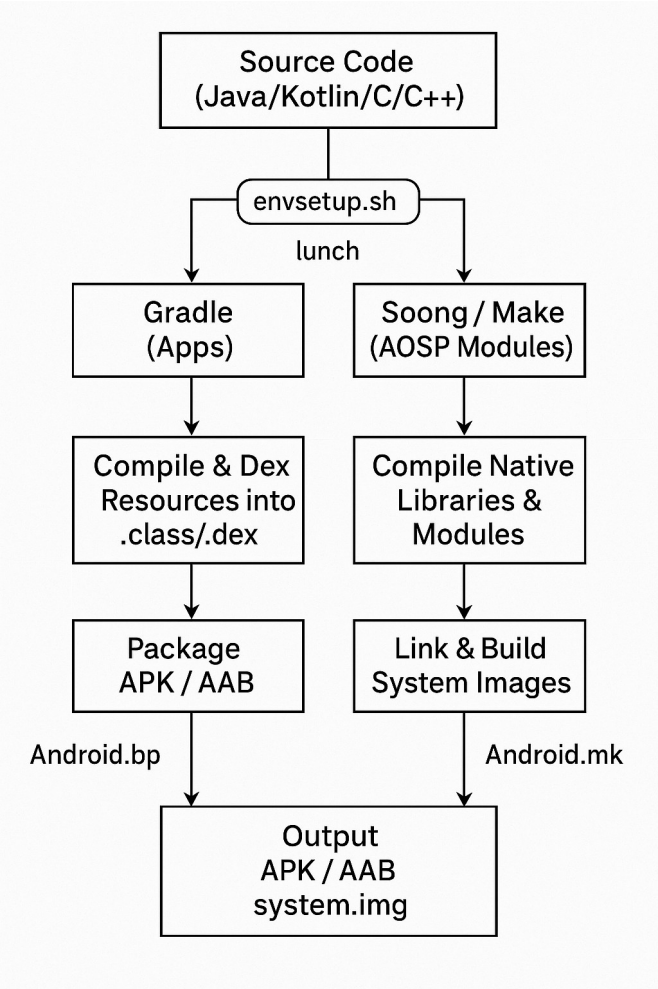
Android Build System



Overview of Android Build System:

- Framework to compile the Android Open Source Project (AOSP) into flashable OS images.
- Handles device diversity, vendor customizations, and maintainability of thousands of projects across hundreds of repositories.
- Compiles code in **Java, C, and C++** into binaries.
- Packages **APKs** and manages resource files.
- Links libraries and produces **bootable system images**.
- Supports **device-specific customizations** and integration of vendor blobs.

- Handles **kernel compilation**.
- For a Build & Release Engineer, mastering this system is crucial.

1. Repo Tool

The Repo tool is a Python-based wrapper on top of Git, created to manage multiple Git repositories in AOSP efficiently using a single manifest.

- Google maintains AOSP across multiple **Git repositories**.
- Cloning repositories one by one is **impractical**.so repo tool was created
- `repo` tool manages multiple Git repositories in a single workspace.
- Uses a **manifest XML** to define all required repositories and branches to use
- Lets you **download and sync all repositories easily**
- Supports **branch management, local changes, and submitting patches** across repos.

2. Build Descriptions (Makefiles & Blueprints)

- The build system needs **metadata** to understand how to compile modules.
- This is provided by files like;
  - **Android.mk** → Makefile format (used in legacy system)
  - **Android**.**bp** → Blueprint format, used in **Soong system** (faster, more maintainable, written in Go).
- Key supporting files:
  - BoardConfig.mk → Hardware-specific settings (partition sizes, kernel path)
  - AndroidProducts.mk → Lists products and their configurations
  - device.mk → Defines device-specific components and dependencies

- **Purpose:** These files tell the build system **what to build, how to build it, and for which device**.

1. Make

- **Definition:** A traditional build automation tool that reads **Makefiles** (`Android.mk`) to determine how to compile and link code.
- **Usage in Android:**
  - Legacy AOSP system uses **Kati** (or `ckati`) to process `Android.mk` files.
  - Converts these instructions into a build plan that can be executed.
- **Output:** System binaries, libraries, APKs, and images.

---

2. Soong

- **Definition:** Modern Android build system written in **Go**, replacing Make for most modules.
- **Input:** Reads **Blueprint files** (`Android.bp`) that describe modules and dependencies.
- **Function:** Generates **Ninja build files** that describe the actual build steps.
- **Advantages:** Faster, more maintainable, and better dependency management than Make.

---

3. Ninja

- **Definition:** A small, fast build system that executes the **build graph** generated by Soong or Kati.
- **Function:** Reads Ninja files and builds the required artifacts in **parallel**, maximizing CPU usage.
- **Output:** Compiled binaries, libraries, APKs, system images, etc.

Summary Flow:

```
Android.mk → Kati → Ninja → Artifacts
Android.bp → Soong → Ninja → Artifacts
```

3. Build Tools

- The Android build system uses **multiple layers of tools** to convert metadata files into compiled artifacts(o/p files produced by build process=APKs, libraries, system binaries)

Tools:

- **Soong** → Parses `Android.bp` files and generates **Ninja build files**.
- **Kati (ckati)** → Converts `Android.mk` files into **Ninja build files** (legacy system).
- **Ninja** → Executes the **build graph**, enabling **fast parallel builds**.

Build Pipeline:

Source files (Android.mk / Android.bp)
↓
Soong / Kati
↓
Ninja
↓
Compiled artifacts (APKs, libraries, system binaries)

4. Environment Setup

- Before starting a build, the environment must be configured using:
    - **source build/envsetup.sh**
- This sets up environment variables and functions like '**lunch**'.
- Lunch selects a build configuration:
    - aosp_arm-eng → Generic ARM emulator build (engineering)
    - -userdebug → Device-specific debuggable build
    - -user → Final production build
- Build variants:
    - eng → Engineering build, root access, extra debugging
    - userdebug → Same as production but debuggable
    - user → Optimized for release, no debugging, secure

5. Build Commands

The build system provides multiple commands:

• make -j$(nproc) → Builds the entire system

This will compile all modules and produce the full system image.

• m → Builds a specific module globally

m Settings

This will build the **Settings app** module only.

• mm → Builds only the current directory module

cd frameworks/base/core

mm

This will build only the modules located in `frameworks/base/core`.

- mmm path/to/module → Builds a module at a given path

  mmm packages/apps/Calculator

  This will build the **Calculator app** module only.

- make bootimage → Builds boot.img only

  This generates `boot.img` without building the entire system.

These allow incremental compilation, saving time for developers.

6. Build Artifacts

- After the build, outputs are stored in:

  out/target/product//
- Key images:

  •boot.img → Contains kernel + ramdisk

  • system.img → Android framework and system apps

  • vendor.img → Vendor-specific HALs, binaries, configs

  • userdata.img → User partition

  • recovery.img → Recovery environment
- These images are later flashed onto the device using fastboot or OTA.

7. Device Customization

- Vendors like Samsung, Motorola, etc., customize AOSP builds for their devices

- Key components:
    - **Device tree** → `device/<vendor>/<device>/`
        - Contains `BoardConfig.mk`, init scripts, and proprietary binaries.
    - **Vendor blobs** → `vendor/<vendor>/`
        - Proprietary files such as drivers, HALs, and firmware.
    - **Kernel sources** → `kernel/<vendor>/<device>/`
        - Device-specific Linux kernel code and configurations.

Summary: Device-specific customizations ensure the AOSP build **works correctly on different hardware** while integrating vendor-specific features.

8. Typical Build Flow

1. **`repo init -u <manifest-url> -b <branch>`** → Initialize the local repo with the desired branch.

2. **`repo sync`** → Download all repositories defined in the manifest.

3. **source build/envsetup.sh** -> Set up environment variables and functions like `lunch`.

4**`.lunch aosp_<device>-userdebug`** → Select a build configuration for the target device.

`5. make -j$(nproc)` → Build the entire system using all CPU cores.

6. **`fastboot flashall`** → Flash the compiled images onto the device.

This process fetches sources, compiles, and flashes to device.

9. OTA & Release Engineering

Build & Release Engineers handle:

- **CVE patch integration** → Apply security fixes per Android security bulletins.
- **Creating OTA packages** → Generate **incremental** and **full updates**.
- **Managing feature branches** → Handle customer-specific customizations.
- **Debugging build issues** → Identify and fix build failures.
- **Automating CI/CD** → Set up automated Android build pipelines.

OTA generation:

- Use `ota_from_target_files` to create OTA packages.
- Deliver updates **securely** to devices.

10. Importance for Build & Release Engineer

- Knowledge of **Git/repo workflows** (branching, patching)
- Understanding **Android.bp** and **Android.mk** files
- **Debugging compilation errors** efficiently
- **Managing multiple device builds**
- **CVE patch backporting and validation**
- **Automating builds** using Jenkins or other CI tools
- Critical for ensuring **timely delivery of secure and stable Android builds**.